Task 3 - Real-time PQC Protection with SPDM

PSoC Control C3M6 (KIT_PSC3M6_EVAL) - SPDM 1.4 responder authenticated with ML-DSA-87, session keys from ML-KEM-1024

1. Summary

The SPDM responder was migrated from the classical profile P2 (ECDSA-P384 / ECDHE-P384) to the CNSA 2.0 profile **P4**: SPDM **1.4**, device identity **ML-DSA-87** (FIPS 204), key establishment **ML-KEM-1024** (FIPS 203), SHA-384 transcript/KDF hash and AES-256-GCM. The device identity key is generated inside the secure world, certified by an ML-DSA-87 root CA (root and leaf both ML-DSA-87, no classical certificate in the chain) and the host requester validates the chain against a pinned ML-DSA-87 root. The responder offers no classical algorithm (BaseAsymAlgo and DHE group are empty), so a classical-only requester or trust anchor cannot negotiate a downgrade.

Area	Result	Key figures (P2 ECC -> P4 PQC)
Positive tests	8/9 PASS (connect, slots, get-cert, challenge, measure, session, 3x encrypted exchange); 1 transport/board fault (fail-closed, see 4.1); plus 2 reconnect exchanges after the tamper images	device session setup 257.2 -> 92.7-320.5 ms (median 138.2; ML-DSA signing time varies)
Negative tests	12/12 REJECTED	modified cert, invalid signature, modified transcript, modified KEM ciphertext, MITM on FINISH / app data, rogue / ML-DSA-65 / classical / malformed anchors
Reconnect after tamper images	PASS	clean image re-flashed, encrypted exchange works
ECC baseline negative tests	6/6 REJECTED on the ECC baseline; clean ECC exchange PASS	same fault hooks on P2: modified cert, invalid ECDSA signature, modified transcript, MITM FINISH / app data, PQC root vs ECC device (4.2b)
Repeat P4 exchanges	3/3 PASS after re-provisioning	checks that the exchange_2 transport fault does not reproduce (4.2c)
Resources	fits: NS RAM 100 % allocated by design (heap = rest), peaks measured	NS .bss 20.0 -> 57.8 KB; S .bss 11.0 -> 18.9 KB; app flash 130.3 -> 127.5 KB
Communication	host-bound (DFU/I2C, 127 B per transaction)	get-cert 17 -> 142 transfers, 1769 -> 18490 B

2. CNSA 2.0 target profile and mapping from the ECC flow

Target: CNSA 2.0 (NSA, CNSSP 15 update) algorithm suite for national-security systems. CNSA 2.0 allows only **ML-DSA-87** for signatures and **ML-KEM-1024** for key establishment (both NIST security category 5, for all classification levels), SHA-384/SHA-512 and AES-256. ML-DSA-65 / ML-KEM-768 (category 3) are therefore **not** CNSA 2.0 compliant even though they are the common commercial default - the brief's warning applies; ML-DSA-65 anchors are actively rejected (test A2). This is a design constraint, not a certification claim.

SPDM function	Original (P2, ECC)	Migrated (P4, PQC)	CNSA 2.0 / standard
Responder authentication (CHALLENGE_AUTH, signed MEASUREMENTS, KEY_EXCHANGE_RSP)	ECDSA P-384 (BaseAsymAlgo)	ML-DSA-87 (PqcAsymAlgo, SPDM 1.4), FIPS 204 context = SPDM op string	ML-DSA-87 required (FIPS 204, cat. 5)
Certificate chain / trust anchor	X.509 P-384 root -> P-384 leaf (1,218 B)	X.509 ML-DSA-87 root (7,578 B) -> ML-DSA-87 leaf (7,557 B); chain 15,187 B	ML-DSA-87 CA signatures
Key establishment (KEY_EXCHANGE)	ECDHE P-384 (DHE group)	ML-KEM-1024 (KEMAlg, SPDM 1.4)	ML-KEM-1024 required (FIPS 203, cat. 5)
Transcript hash / HKDF / HMAC	SHA-384	SHA-384	SHA-384 or SHA-512
Session AEAD	AES-256-GCM	AES-256-GCM	AES-256
Measurement hash	SHA-384	SHA-384	SHA-384 or SHA-512
SPDM version	1.2/1.3 (classic algorithms)	1.4 (PQC fields)	DSP0274 1.4, DSP2072

No hybrid ECC+PQC mode was built (excluded by the brief). The migration is a clean profile switch: both profiles share the same responder source; `SPDM_PROFILE=P4` selects the algorithm set at build time, so a P4 image cannot fall back to ECC at run time.

3. Migrated architecture

3.1 How the host validates the PQC identity

1) VERSION/CAPABILITIES/ALGORITHMS: the requester negotiates SPDM 1.4 and sees PqcAsymAlgo=ML_DSA_87, KEMAlg=ML_KEM_1024, BaseAsym=none, DHE=none. 2) GET_DIGESTS / GET_CERTIFICATE: the 15,187 B SPDM cert_chain (header, SHA-384 RootHash, root DER, leaf DER) is fetched in 7,168 B portions (max SPDM message) which the transport splits into 127 B CHUNK_GET transfers. 3) The requester checks that the first certificate equals the pinned root (DER), verifies the leaf's ML-DSA-87 CA signature and the RootHash. 4) CHALLENGE_AUTH / KEY_EXCHANGE_RSP carry a 4,627 B ML-DSA-87 signature over the SPDM 1.2+ signing context (100 B prefix + transcript hash) with the op-code string as FIPS 204 context; the requester verifies it with the leaf public key. Only after this does the requester trust measurements or a session.

3.2 How the session secret is established and derived

KEY_EXCHANGE (1,638 B) carries the requester's ephemeral ML-KEM-1024 encapsulation key (1,568 B). The responder's non-secure libspdm hands it through an NSC veneer to the secure world, which runs ML-KEM-1024.Encaps (wolfCrypt) and returns ciphertext (1,568 B) and the 32 B shared secret. KEY_EXCHANGE_RSP (6,305 B = ciphertext + ML-DSA-87 signature + ResponderVerifyData + nonce/opaque) is signed over the transcript TH1 (VCA + cert-chain hash + KEY_EXCHANGE + RSP). Both sides run the SPDM key schedule (DSP0277/DSP0274: HKDF-SHA-384 handshake secret from the KEM shared secret, bound to TH1) -> request/response handshake keys and finished keys; ResponderVerifyData (HMAC) and the encrypted FINISH/FINISH_RSP confirm the keys (TH2), then the master secret yields the AES-256-GCM data keys. Decapsulation happens on the host; the device never holds a long-term KEM key, so each session has fresh, forward-secret keys. ML-KEM implicit rejection means a modified ciphertext gives an unrelated secret, which key confirmation detects (test B4).

3.3 Credential and trust-anchor provisioning / update

Provisioning app (ML-DSA-87 suite, `PROV_SIG_SUITES=ml-dsa-87`) over the same I2C/DFU link: *gen-key* creates the ML-DSA-87 key pair inside the SPE (PAS key slot 0, private key never exported) -> *get-csr* (CSR self-signed with ML-DSA-87, 7,355 B) -> OEM CA signs (`sign_csr.py`, ML-DSA-87 root) -> *put-cert --spdm-chain --hash-alg sha384* into PAS cert slot 1 -> read-back + *prove-cert* (device proves possession of the key for the stored leaf). Script: `tools/task3_provision_p4.ps1` (41 s incl. on-device key generation). The slots stay re-writable because `--freeze` was not used on the shared board; in production, freeze after provisioning and rotate by provisioning a second slot. Host side: the trust anchor is the pinned root DER; rotating the CA means distributing a new pinned root. The responder reads the chain from PAS at start-up, so a credential update needs no new firmware.

3.4 How the large PQC objects fit the MCU memory model

Object	Size	Where / how bounded
Cert chain (root + leaf + header)	15,187 B (P2: 1,218 B)	PAS cert slot (15 KB, 99 % full) and a 16 KB NS RAM copy (LIBSPDM_MAX_CERT_CHAIN_SIZE)
ML-DSA-87 signature	4,627 B	built in the SPE output buffer, copied into the response
ML-DSA-87 private key object (wolfCrypt)	7,912 B static (S RAM)	private key never leaves the SPE
ML-KEM-1024 encap key / ciphertext	1,568 B / 1,568 B	inside KEY_EXCHANGE / _RSP buffers
Largest SPDM message	7,168 B (CERTIFICATE portion); KEY_EXCHANGE_RSP 6,305 B	LIBSPDM_MAX_SPDM_MSG_SIZE 7 KB (P2: 2 KB); messages > 127 B use CHUNK_SEND/CHUNK_GET
libspdm scratch (message + chunk reassembly)	35,843 B static (NS RAM)	fixed size, checked against libspdm_get_sizeof_required_scratch_buffer() at init
libspdm context	3,328 B	1 session, 1 root (MAX_SESSION_COUNT=1)
Transcript	hash only	LIBSPDM_RECORD_TRANSCRIPT_DATA_SUPPORT=0: running SHA-384, not message copies
Secure hash pool / heap peak	9,280 B / 27,720 B	wolfCrypt ML-DSA/ML-KEM working memory in the SPE heap

All buffers are static and size-checked; no allocation grows with message size. Larger chains (e.g. an intermediate CA) do not fit the 15 KB PAS cert slot - a documented constraint.

4. Test results (hardware)

Positive suite: `tools/task1_spdm_runs.ps1 -Profile P4 -Tag task3` -> `results/task3_P4_20261111_1449`. Requester = supplied spdm-requester.exe 2.1.0, pinned root = ca_mldsa87 (DER). Device log: [PERF] UART lines.

Test	Operation	Result	Host wall clock
connect	VERSION/CAPABILITIES/ALGORITHMS (SPDM 1.4, ML-DSA-87, ML-KEM-1024)	PASS	5.6 s
slots	GET_DIGESTS	PASS	6.9 s
get_cert	chain fetch + verify to pinned ML-DSA-87 root	PASS	212.3 s
challenge	CHALLENGE_AUTH, ML-DSA-87 signature	PASS	278.3 s
measure	signed GET_MEASUREMENTS	PASS	290.9 s
session	KEY_EXCHANGE (ML-KEM-1024) / FINISH / END_SESSION, AES-256-GCM	PASS	375.2 s
exchange_1	encrypted application message echo	PASS	344.8 s
exchange_2	encrypted application message echo	FAIL - transport (4.1)	351.8 s
exchange_3	encrypted application message echo	PASS	371.6 s

4.1 Transport fault during exchange_2

exchange_2 failed with `SPDM_REQ_ESESSION`. Evidence: transfers 1-211 completed (chain verified, ML-KEM KEY_EXCHANGE processed and signed on the device, 54 of 55 KEY_EXCHANGE_RSP chunks delivered). For transfer 212 (last CHUNK_GET) the requester received an SPDM **1.0** ERROR (UnsupportedRequest) - the version a responder uses when it has no negotiated state, i.e. consistent with a responder reset. At the same instant (15:20:58) the PC-side UART capture of the board's debug COM port stopped and recorded nothing more for the rest of the suite. Windows logged no USB/PnP event, so the root cause (board reset / USB power glitch vs. lost I2C transaction) could not be determined. Security-relevant outcome: the handshake was aborted and no session was established - the protocol failed **closed**. exchange_3, the 11 negative tests and the reconnect runs R1/R2 (3 board re-flashes) all behaved as expected afterwards; the device metrics for exchange_3 / A0 are missing because of the stopped capture. Engineering note: a P4 session needs ~215 transport transactions vs ~25 for P2, so it is ~9x more exposed to any transport fault; a production requester should retry/resume chunk transfers instead of aborting, and a larger-MTU transport (MCTP over I3C / PCIe VDM) reduces the exposure. Reproducibility: after re-provisioning, 3/3 PASS further P4 exchanges/sessions (4.2c) plus E1a/E1b/E2b (4.2) completed without a transport fault, i.e. the fault did not reproduce in 6 later full handshakes.

Negative tests

Part A: host-side trust anchors (genuine responder). Part B: responder images built with `SPDM_FAULT_INJECT=<mode>` (test-only build flag, `proj_cm33_ns/spdm/spdm_fault_inject.h`, default 0 = compiled out); each image logs '[FI] ...' on UART when it tampers. Runner: `tools/task3_neg_tests.ps1` -> `results/task3_NEG_P4_20261111_1533`.

#	Tampering	Expected	Result	Requester evidence
A0 (suite)	pinned root: classical ECDSA P-384 CA (ca_ec384)	reject	REJECTED	exit=1
A1	pinned root: ML-DSA-87 CA with the same subject name, different key (impersonation)	reject	REJECTED	get_certificate failed: SPDM_REQ_EAUTH
A2	pinned root: ML-DSA-65 CA (non-CNSA parameter set), same subject	reject	REJECTED	get_certificate failed: SPDM_REQ_EAUTH
A3	pinned root: classical ECDSA P-384 CA (classical downgrade of the anchor)	reject	REJECTED	get_certificate failed: SPDM_REQ_EAUTH
A4	pinned root: genuine root with 1 byte flipped in the ML-DSA-87 public key	reject	REJECTED	get_certificate failed: SPDM_REQ_EAUTH
A5	pinned root: genuine root truncated to half (malformed DER)	reject	REJECTED	get_certificate failed: SPDM_REQ_EAUTH
B1	device serves chain with leaf CN changed (modified on-device certificate)	reject	REJECTED	get_certificate failed: SPDM_REQ_EAUTH
B2	device flips 1 byte of its ML-DSA-87 signature	reject	REJECTED	challenge failed: SPDM_REQ_EAUTH
B3	device signs a transcript hash with 1 bit flipped (modified transcript)	reject	REJECTED	start_session failed: SPDM_REQ_ESESSION
B4	ML-KEM ciphertext modified before signing (handshake data altered)	reject	REJECTED	start_session failed: SPDM_REQ_ESESSION
B5	MITM flips AEAD tag of FINISH_RSP (modified handshake message)	reject	REJECTED	start_session failed: SPDM_REQ_ESESSION
B6	MITM flips AEAD tag of the encrypted application data	reject	REJECTED	send_receive failed: SPDM_REQ_ESESSION
R1	genuine image re-flashed, reconnect + encrypted exchange	connect	PASS	RESULT: PASS

#	Tampering	Expected	Result	Requester evidence
R2	reconnect + encrypted exchange (repeat)	connect	PASS	RESULT: PASS

Note: a third repeat reconnect (R3) was interrupted by the host tooling (the background runner was stopped because of PC memory pressure at transfer 181 of ~215); it produced no result and is not counted. All required tests had completed before.

B3-B6 were checked against the device log to rule out a transport loss: the complete KEY_EXCHANGE_RSP (all 55 chunks) was delivered and the device logged the injected fault ('[FI] ...') before the requester aborted. B4 shows why key confirmation matters with a KEM: the signature over the (modified) ciphertext is valid, but ML-KEM implicit rejection gives the requester a different secret, so ResponderVerifyData fails.

4.2 Edge cases (hardware)

Runner `tools/task3_edge_tests.ps1 -Phase P4` (UART capture now re-opens the port and logs capture errors, which hid the exchange_2 evidence). Results `results/task3_EDGE_P4_20261111_1705`, `task3_EDGE_P4_20261111_1729`.

Test	Scenario	Expected	Result	Evidence
E1a	repeat session (stability)	pass	PASS	RESULT: PASS
E1b	repeat session (stability)	pass	PASS	RESULT: PASS
E2	requester killed during KEY_EXCHANGE_RSP transfer (abandoned handshake)	done	PASS	requester killed during KEY_EXCHANGE_RSP transfer
E2b	reconnect + encrypted exchange right after the abandoned handshake	pass	PASS	exchange: OK (encrypted app message, sent 23 B, received 23 B)
E3	get-cert from unprovisioned slot 1	reject	REJECTED	spdm-requester.exe : error: slot 1 is empty (slot mask 0x01; populated slots: 0)
E5	J-Link target reset, then CHALLENGE (identity persists in PAS)	pass	PASS	RESULT: PASS

Not tested (dropped to save board time, ~5 min per SPDM operation): application-message size limits (E4: 90 B / 400 B echo); provisioning-side rejections (unsupported key algorithm, occupied slot, oversize chain, leaf not matching the device key, SHA-256 RootHash) - moved to Task 4 (credential update).

4.2b ECC baseline (P2) negative tests

Same fault-injection hooks (profile-independent) built with `SPDM_PROFILE=P2` against the provisioned ECDSA P-384 identity (mode 4 is ML-KEM only). Closes the Task 1 open item: tampering is rejected in both the baseline and the migrated design. Results `results/task3_EDGE_P2FI_20261111_1753`.

Test	Scenario	Expected	Result	Evidence
P2_B1_modified_cert	ECC device serves chain with leaf CN changed	reject	REJECTED	spdm-requester.exe : error: get_certificate failed: SPDM_REQ_EAUTH
P2_B2_invalid_signature	ECC device flips 1 byte of its ECDSA P-384 signature	reject	REJECTED	spdm-requester.exe : error: challenge failed: SPDM_REQ_EAUTH
P2_B3_modified_transcript	ECC device signs a transcript hash with 1 bit flipped	reject	REJECTED	spdm-requester.exe : error: start_session failed: SPDM_REQ_ESESSION
P2_B5_mitm_finish_rsp	MITM flips AEAD tag of FINISH_RSP (ECDHE session)	reject	REJECTED	spdm-requester.exe : error: start_session failed: SPDM_REQ_ESESSION
P2_B6_mitm_app_data	MITM flips AEAD tag of the encrypted application data (ECDHE session)	reject	REJECTED	spdm-requester.exe : error: send_receive failed: SPDM_REQ_ESESSION
P2_R1_clean_exchange	clean ECC image: session + encrypted exchange (positive)	pass	PASS	exchange: OK (encrypted app message, sent 8 B, received 8 B)
P2_R2_pqc_root_on_ecc_device	requester pins the ML-DSA-87 root, device holds the ECC identity	reject	REJECTED	spdm-requester.exe : error: get_certificate failed: SPDM_REQ_EAUTH

4.2c Restore of the PQC identity and repeat exchanges

After the P2 tests the ML-DSA-87 identity was provisioned again (new on-device key, same ca_mldsa87 root, SHA-384 RootHash) and the clean P4 image re-flashed; three further encrypted exchanges check whether the exchange_2 transport

fault (4.1) reproduces. Results `results/task3_EDGE_RESTORE_20261111_1805`.

Test	Scenario	Expected	Result	Evidence
R_restore_exchange	P4 identity re-provisioned, clean P4 image: session + encrypted exchange	pass	PASS	exchange: OK (encrypted app message, sent 11 B, received 11 B)
R_repeat_exchange_2	repeat P4 exchange (exchange_2 reproducibility)	pass	PASS	exchange: OK (encrypted app message, sent 8 B, received 8 B)
R_repeat_exchange_3	repeat P4 exchange (exchange_2 reproducibility)	pass	PASS	exchange: OK (encrypted app message, sent 8 B, received 8 B)
R_restore_get_cert	get-cert of the restored ML-DSA-87 chain	pass	PASS	wrote certificate chain to C:\Users\kpato\KP\Hackathon\results\task3_E

4.3 Automated verification

`tools/tests/test_task3_offline.py` (unittest, no board): 50 checks - SPDM cert_chain structure (length, SHA-384 RootHash, root+leaf ML-DSA-87, signatures verify, validity, fits 16 KB libspdm buffer and 15,360 B PAS slot, device-served chain = provisioned blob), each negative trust anchor is what it claims (rogue key, ML-DSA-65, P-384, one byte inside the public key, unparseable), fault-injection hooks all guarded and off by default, deployed image contains no [FI] code, patch applies to the base commit and reproduces the working tree, every recorded rejection is a protocol error (EAUTH/ESESSION) and - for handshake faults - happened after the complete KEY_EXCHANGE_RSP was delivered with the matching [FI] line logged, message sizes consistent with ML-KEM-1024 / ML-DSA-87 sizes, measurement parser incl. partial UART lines. Build matrix (RUN_BUILD_TESTS=1): every SPDM_FAULT_INJECT=1..6 image contains only its own hook, FI=0 none - PASS. Found and fixed by the tests: measurement script crashed on a partial UART line and ran its CLI on import.

5. Measurements - P2 (ECC baseline, Task 1) vs P4 (PQC)

Same board, Release (-Os) responder with [PERF] logging, NS SysTick timer, I2C 400 kHz via KitProg3. Times are device-side processing per message; host wall clock is dominated by dfuh-cli start-up (~1.4 s per 127 B transaction).

5.1 Performance (device)

Operation	P2 ECDSA/ECDHE P-384 [ms]	P4 ML-DSA-87 / ML-KEM-1024 [ms]	Comment
Signature generation (SPE)	84.3 - 84.5 (median 84.4, n=6)	66.0 - 346.2 (median 112.0, n=5)	ML-DSA signing is rejection-sampled -> variable time
Key exchange: ECDHE keygen+shared / ML-KEM encaps	162.9 (80.7 + 82.2)	11.1 - 11.2 (median 11.1, n=3)	ML-KEM ~15x faster than SW P-384 ECDH
KEY_EXCHANGE message total	252.1 - 252.5 (median 252.2, n=4)	86.8 - 314.6 (median 132.3, n=3)	sign + KEX + transcript
FINISH (secured)	5.0 - 5.1 (median 5.0, n=4)	5.9 - 5.9 (median 5.9, n=2)	HMAC verify + key derivation
Encrypted app message round (AEAD)	1.15 - 1.23	1.9 - 1.9 (median 1.9, n=3)	AES-256-GCM, small message
Signature verification on device	n/a	n/a	responder does not verify (no mutual auth)
Decapsulation	n/a	n/a (host side)	responder is the encapsulator

5.2 Communication

Metric	P2	P4	Delta
Cert chain size	1,218 B	15,187 B	x12.5
Largest SPDM message	1,226 B (CERTIFICATE)	7,168 B (CERTIFICATE portion); KEY_EXCHANGE_RSP 6,305 B	
KEY_EXCHANGE request / response	166 B / 302 B	1,638 B / 6,305 B	x9.9 / x20.9
get-cert: transfers / wire bytes	17 / 1,769 B	142 / 18,490 B	
session (VCA + cert + KEY_EXCHANGE + FINISH + END): transfers / bytes	24 / 2,569 B	214 / 28,199 B	
Fragmentation	CHUNK_SEND x2 / CHUNK_GET x3 for KEX	CHUNK_SEND x14 / CHUNK_GET ~50 for KEX; cert 3 portions x ~57 chunks	127 B DFU payload limit

Metric	P2	P4	Delta
Host wall clock per 'session'	38.3 s	375.2 s	host-bound (dfuh-cli per transfer)
Device busy per 'session'	265.9 ms	360.7 ms	incl. cert-chain serving

5.3 Memory (static from build map, Release without logging; peaks from [PERF])

Region	P2	P4	Delta / note
Secure flash (128 KB)	82,300 B	79,532 B	-2.8 KB (wolfCrypt PQC < SW P-384 mbedTLS)
Non-secure flash (160 KB)	48,036 B	47,972 B	-64 B
Secure .bss	11,016 B	18,940 B	+7.9 KB (ML-DSA key object, hash pool)
Non-secure .bss	19,952 B	57,816 B	+37.9 KB (16 KB chain copy, 35.8 KB scratch)
NS heap region left	44,600 B	6,736 B	tight but sufficient (peak NS heap 1,032 B)
Peak NS stack / S stack	2,328 / 4,336 B	2,328 / 2,608 B	of 4 KB / 8 KB
Peak S heap	2,576 B	27,720 B	of 31 KB S heap region
Credential store (PAS)	1.2 KB used of 24 KB	15.2 KB cert + ML-DSA-87 key (8 KB slot)	cert slot 99 % full
Peak handshake RAM (static NS+S .bss + peaks)	~40 KB	~108 KB	of 126 KB S+NS SRAM

6. Remaining classical dependencies and limitations

Item	Status	What must change
BootROM / FLASH_BOOT	classical (ECDSA OEM key); not enabled on the shared board (Task 1/2)	silicon/ROM with PQC or EPBL anchored via OEM provisioning + EPSS-signed certificate
Boot -> SPDM identity link	SPDM key is a separately provisioned credential (PAS); measurements report firmware digests	bind identity to verified boot (Task 4: DICE-style / measurement of EPBL-verified image)
Provisioning lock	--freeze not used (shared board) -> slots re-writable by the provisioning app	freeze + lifecycle SECURE in production
Mutual authentication	not used (requester is not authenticated)	MUT_AUTH with requester ML-DSA-87 cert
Transport	127 B DFU transactions over I2C; one unexplained transport/board fault aborted one handshake (fail-closed)	larger MTU transport and CHUNK_GET retry
Requester tool	re-fetches the 15 KB chain on every operation (~3.5 min host time)	cache chain by GET_DIGESTS slot digest; faster transport than DFU-per-127-B
Cert slot capacity	15 KB slot holds root + leaf only	larger PAS slot or store only leaf + root hash for intermediate CAs
ML-DSA signing time	variable (rejection sampling, 111-350 ms)	acceptable; budget worst case
Fault-injection builds	test-only images (SPDM_FAULT_INJECT != 0) print a banner	never ship; default 0 compiles hooks out

7. Evidence and reproduction

```
Build provisioning app : make -C psc3-provisioning-app build   (common.mk PROV_SIG_SUITES ?= ml-dsa-87)
Program                : make -C psc3-provisioning-app qprogram  (no full erase)
Provision P4 identity  : tools/task3_provision_p4.ps1           (no --freeze)
Build responder P4     : make -C psoc-spdm-responder clean build SPDM_PROFILE=P4 ; qprogram
Positive suite         : tools/task1_spdm_runs.ps1 -Profile P4 -Tag task3
Negative anchors       : tools/task3_make_neg_roots.py ; tools/task3_neg_tests.ps1
Measurements           : tools/perf_summary.py <run dir>
Report                 : py -3.13 -I tools/make_task3_report.py
Positive run           : results/task3_P4_20261111_1449
Negative run           : results/task3_NEG_P4_20261111_1533
Logs                   : results/task3_prov_build.log, task3_provision_p4.log, task3_resp_p4_build.log
```